



FOR THE POET AND THE CRITIC

Poetry with Inkhaven

Reading, Measuring, and Making Verse

Vladimir Ulogov

2026 · examples assume Inkhaven 1.8.17 or newer

Contents

Two readers, one workbench	1
What you need	2
Part I — The Line	3
1 — Verse in Inkhaven	4
The verse paragraph family	5
Declaring a form: the poem: block	6
The forms library	7
2 — The sound of a line	11
Counting syllables	12
Finding the stress	13
Scanning a whole line	14
Part II — Measure	16
3 — Metre and scansion	17
The vocabulary of the foot	18
Scanning a line	18
Why English fights back — and the accent mark	20
Scanning against a declared form	21
The three traditions of measure	22
4 — Rhyme	24
Asking whether two words rhyme	25
The four qualities of rhyme	25
Masculine, feminine, dactylic	26
Every language rhymes differently	27
Part III — The Whole Poem	29
5 — The Inner Poet	30
Two speeds of reading	31
The fast track: findings	31
Running the fast track in the editor	32
The slow track: engaging the Poet	33
6 — Form and completion	35
How far along are you?	36
Structural issues	37
Status in the workflow	38
Part IV — Beyond the Original	40

7 — Translating verse	41
The verse-translation paragraph	42
The trilemma, scored	42
Reading a translation as a critic	44
8 — Scripting and scholarship	45
Machine-readable output: --json	46
The engines in Bund	47
Where the workbench ends	49
Part V — At the Desk	50
9 — Writing a poem at the desk	51
Make a stanza	52
Declare the form	53
Write, and watch the count	54
Read the poem back	54
Let it read as you go — ambient	55
The whole book at a glance	55
Source beside translation	56
The loop, entire	57
Appendix A — The poetry workflow	58
Forms — declaring intent	58
Measuring — sound, metre, rhyme	59
The Inner Poet — reading a stanza	59
Translation	60
In the editor	60
In Bund	60
About the Author	62
A work of love	63
A note on cooperation	63
Where to find more	64

Before we begin

There is a thing this book will never do, and it is worth saying on the first page so that no one waits for it: **Inkhaven does not write poems.** No command produces a line of verse. No key summons a stanza. If you came hoping to type a topic and receive a sonnet, close the book gently and go elsewhere; you will not find that here, by design.

What Inkhaven does — the only thing every poetry feature does — is **observe, measure, and report.** It counts your syllables. It scans your line and names the metre it hears. It tells you whether two of your endings truly rhyme, and in which tradition, and how. It checks a villanelle for its refrains and a sonnet for its turn. It weighs a translation against its original on form and sound. And then it stops, and hands the verdict back to you, and says nothing about what you should do next. The poem is yours. The judgement is yours. Inkhaven is the ruler, the tuning fork, and the honest second reader — never the author.

Two readers, one workbench

The title of this book drops a word on purpose. It is not **Writing Poetry with Inkhaven**, because the workbench serves two people who are often the same person on different days.

The first is the **poet** — someone making verse, who wants a second reader that never flatters. You declare that this is a sonnet in iambic pentameter; Inkhaven holds you to it, line by line, and marks where the foot slips or the rhyme goes slack — not to correct you (a dropped foot is often the best thing in a poem) but so that when you break a rule you are **choosing** to, with your eyes open.

The second is the **critic** — the scholar, the student, the translator, the teacher — someone writing **about** verse that already exists. You paste in a stanza of Pushkin or Shakespeare and ask the workbench to scan it, classify its rhymes, name its form, or measure how faithfully a translation carried the sound across. Here Inkhaven is an instrument of close reading: it makes the mechanics of a poem explicit

and countable, so your argument about the poem rests on evidence rather than impression.

The same commands serve both. `poetry metre` does not care whether the line it scans is yours or Milton's. That neutrality is the whole point — and it is why this book teaches the measuring first and leaves the making, always, to you.

A NOTE ON THE EXAMPLES

The worked examples lean on poems in the public domain — Shakespeare's sonnets, Pushkin's tetrameter, a little Rilke and Verlaine — because they are famous, fixed, and free to quote. Every feature is also shown across Inkhaven's five prose languages (English, Russian, French, German, Spanish); where a technique behaves differently in Russian than in English, we stop and look, because those seams are where the prosody actually lives.

What you need

You do not need to be a metrist, and every term is defined where it first appears — a *foot*, a *volta*, a *feminine ending* are all explained on the page that first needs them. You do not need to read Russian; every non-English example is translated. What you do need is a working copy of Inkhaven (the examples assume 1.8.17 or newer) and a willingness to let a poem be measured — to hear a machine count what you felt, and to keep your own counsel about what the count means.

A word on honesty, carried over from this book's companions. Measuring verse from plain text has hard edges — English rhyme and metre, read from spelling alone with no pronouncing dictionary, are rougher than Russian's, where the orthography wears its sounds on its sleeve. This book does not hide those edges. Where the tool guesses, it says so, and so will we.

POETRY WITH INKHAVEN

PART I

The Line

CHAPTER 1

1

Verse in Inkhaven

A poem is not a paragraph of prose with the line breaks left in. It has structure that prose does not — the line is a unit, the stanza is a unit, and both carry meaning that would be destroyed by reflowing the text to fit the page. Before Inkhaven can measure verse, it has to **hold** verse: know that this block is a line and that one a quatrain, and preserve those boundaries all the way to the printed page. This chapter is about how it does that, and how you tell it that a given passage is a poem at all.

The verse paragraph family

Inkhaven does not have a separate "poetry project" with its own hierarchy. A book of verse is an ordinary Inkhaven book; poems live inside it as paragraphs of a special kind. Every paragraph in Inkhaven carries a **structural type** — a short tag that says what the paragraph *is* — and poetry adds a family of them, all prefixed `para:verse-`:

Structural type	Glyph	What it holds
<code>para:verse-line</code>		a single line of verse
<code>para:verse-stanza</code>	♪	a stanza of any length — the workhorse
<code>para:verse-couplet</code>	=	a two-line unit
<code>para:verse-tercet</code>	:	a three-line unit
<code>para:verse-quatrain</code>	∴	a four-line unit
<code>para:verse-translation</code>	⇔	a translated stanza, paired with its source

The glyphs are not decoration — they are how the Tree and Outline panes show you the shape of a poem at a glance. A sequence of ♪ ♪ ♪ ♪ in the outline is four stanzas; a run of ∴ is a poem built in quatrains. You are never guessing at structure you cannot see.

TERM Structural type

The tag on every Inkhaven paragraph naming its role — `para:heading`, `para:body`, `para:verse-stanza`, and so on. It is metadata, not markup: it travels with the paragraph in the store and drives how the paragraph is displayed, measured, and exported, without putting anything visible into your text.

Which one should you reach for? Ninety per cent of the time, `para:verse-stanza`. It holds a stanza of any length and is what the measuring tools expect. The fixed-size units — couplet, tercet, quatrain — earn their keep when a poem's form is built from them (a poem **in** quatrains, a sonnet's closing couplet) and you want the outline to say so. `para:verse-line` is for the rare case where a single line stands alone. And `para:verse-translation` is the subject of Chapter 7.

LINE BREAKS ARE PRESERVED

Inside a verse paragraph, the newlines you type are the newlines that print. Inkhaven does not reflow verse the way it reflows prose. A line is a line; where you break it is where it breaks. This is the one place in the editor where whitespace is load-bearing.

Declaring a form: the poem: block

A stanza sitting on its own is just text with line breaks. What turns it into something Inkhaven can **hold you to** is a declaration of intent — a statement that says "this is meant to be a sonnet, in iambic pentameter, rhyming ABAB CDCD EFEF GG." That declaration is a **poem block**: a small HJSON record attached to a poem (as a sidecar, or inline) that names its form.

Here is the block for a sonnet, exactly as `inkhaven poetry forms --form sonnet` prints it — an HJSON record, one field per line:

```
poem: {
  // 14 lines of iambic pentameter (generic English scheme)
  form: sonnet
  metre: iambic
  feet: 5
  metre_tradition: accentual_syllabic
  rhyme_scheme: "ABAB CDCD EFEF GG"
  language: en
}
```

Every field is a promise you are making about the poem, and every measuring tool in this book reads them:

- **metre and feet** — the accentual-syllabic target: `iambic` × 5 is iambic pentameter, the ten-syllable line. Chapter 3 scans against this.
- **metre_tradition** — which system of measure applies. `accentual_syllabic` (English, German), `syllabic` (French and the Japanese forms, counting syllables only), or `accentual` (counting stresses only). The engine measures the right thing for the tradition.

- **rhyme_scheme** — the pattern of line endings; matching letters must rhyme. Chapter 4 checks it.
- **stanzas and lines_per_stanza** — the architecture the completion tools in Chapter 6 hold you to. A form prints them only when it fixes them: a haiku's block carries `stanzas: 1` and `lines_per_stanza: 3`; the open sonnet block above leaves them unstated.

Why declare it at all?

Because measurement needs a target. "Is this line the right length?" has no answer until you have said what length is right. The poem block is where you say it. Declare iambic pentameter and a slipped foot becomes a **finding**; declare nothing and the same line is just a line. The declaration is what turns a passive ruler into an active second reader — and, crucially, you are the one who writes it. Inkhaven never guesses your intent and never imposes a form. It measures only against the promises you chose to make.

The forms library

You rarely write a poem block from scratch, because Inkhaven ships a library of the forms poets actually use. `inkhaven poetry forms` lists them:

```

$ inkhaven poetry forms
poetry forms - `--form <name> [--language en|ru|fr|de|es]` prints
a poem: block:

    sonnet                14 lines of iambic pentameter (generic
English scheme)
    petrarchan_sonnet     Italian sonnet: octave (ABBAABBA) +
sestet (CDECDE)
    shakespearean_sonnet English sonnet: 3 quatrains + a
couplet, volta near line 12
    haiku                 3 lines, 5-7-5 syllables, unrhymed
    senryuu               haiku form (5-7-5) on human nature
rather than the seasons
    tanka                 5 lines, 5-7-5-7-7 syllables, unrhymed
    ghazal                autonomous couplets sharing a radif; a
signature in the closing maqta
    villanelle            19 lines: 5 tercets + a quatrain, two
refrains on a schedule
    pantoum               interlocking quatrains: lines 2 and 4
recur as 1 and 3 of the next
    terza_rima             chained tercets: ABA BCB CDC ...
    ottava_rima            8-line stanzas rhyming ABABABCC
    blank_verse            unrhymed iambic pentameter
    heroic_couplet        rhymed iambic-pentameter couplets (AA
BB ...)
    limerick              5 anapestic lines rhyming AABBA (long-
long-short-short-long)
    ode                   elevated address; stanzaic but formally
open
    elegy                 reflective lament; formally open, often
iambic
    free_verse             no metre or rhyme – the Inner Poet
observes tendencies only
    prose_poem            poetic prose without line breaks

```

There are eighteen in all — the fixed forms of the European tradition (sonnets, the villanelle, terza and ottava rima), the syllabic forms of the Japanese (haiku, senryū, tanka), the ghazal from the Persian and Urdu, and open forms (ode, elegy, free verse, prose poem) for when you want the tools without the corset. To use one, print its block and paste it onto your poem:

```
$ inkhaven poetry forms --form villanelle
poem: {
  // 19 lines: 5 tercets + a quatrain, two refrains on a schedule
  form: villanelle
  metre: iambic
  feet: 5
  metre_tradition: accentual_syllabic
  rhyme_scheme: "ABA ABA ABA ABA ABA ABAA"
  language: en
  stanzas: 6
}
```

Forms speak five languages

The `--language` flag does more than stamp a label. A form carries language-specific prosodic defaults, because the **same** form behaves differently in a different tongue. Ask for a sonnet in Russian —

```
$ inkhaven poetry forms --form sonnet --language ru
poem: {
  ...
  language: ru
  allow_pyrrhic: true
  require_final_stress: true
}
```

— and the block comes back with `allow_pyrrhic` and `require_final_stress` set, encoding the conventions of the Russian iambic line (its tolerance for a stress-light foot, its insistence on a stressed final syllable). This is the multilingual promise the whole workbench keeps: a feature that only worked for English would be a feature that only half-worked. Every form, every measure, is meant to hold in all five languages, and where it cannot, the tool tells you rather than pretending.

Rolling your own

When no built-in fits — an invented form, a nonce stanza, a house style — scaffold a custom block:

```
$ inkhaven poetry forms --new --name my-triolet
// A custom-form scaffold – edit the fields, then paste this into
a
// `poem:` sidecar, or into .inkhaven/custom-forms.hjson to reuse
it.
poem: {
  title: "my-triolet"
  form: custom
  metre: iambic
  feet: 5
  metre_tradition: accentual_syllabic
  rhyme_scheme: "ABAB"
  language: en
}
```

Edit the fields to describe your form, drop it on a poem, and every tool in this book measures against it exactly as it would a built-in. The library is a starting point, not a fence.

WHAT YOU LEARNED

- A book of verse is an ordinary Inkhaven book; poems live in it as paragraphs of the `para:verse-*` family — stanza, couplet, tercet, quatrain, line, translation — each with a glyph that shows the poem’s shape in the outline.
- `para:verse-stanza` (J) is the workhorse; the fixed-size units earn their place when a poem is **built** from them and you want the structure visible.
- Line breaks inside a verse paragraph are load-bearing — Inkhaven preserves them instead of reflowing.
- A **poem block** declares a poem’s intended form — metre, feet, tradition, rhyme scheme, architecture. Measurement needs this target; you write it, Inkhaven never guesses it.
- `inkhaven poetry forms` ships eighteen forms, localised to all five languages; `--new` scaffolds a custom one. Every measuring tool reads the block the same way.

CHAPTER 2

2

The sound of a line

Everything else in this book stands on one operation: cutting a word into syllables and finding which one is stressed. Metre is a pattern of stressed and unstressed syllables; rhyme is an agreement of sounds from the stressed syllable onward; a haiku is a count of syllables and nothing else. Get the syllables wrong and every measurement above them is wrong too. So we begin at the bottom, with the humblest and most consequential tool in the workbench: `poetry syllabify`.

Counting syllables

Ask Inkhaven to break a word:

```
$ inkhaven poetry syllabify extensive
    extensive          ex·'ten·sive (3 syl)
```

Three things happened. The word was **cut** into syllables at `ex·ten·sive`; the **count** came back as three; and the stressed syllable was marked with `'` before `ten`. Each of those is a separate act of judgement, and it is worth knowing how each is made, because each has failure modes a poet needs to see coming.

Where the cuts fall

The syllable boundaries come from *hypher*, the same hyphenation engine Typst uses to break words at the end of a line — which means Inkhaven inherits a mature, per-language hyphenation dictionary without adding a single dependency. Hyphenation and syllabification are not quite the same thing (hyphenation is conservative — it would rather miss a break than put one in the wrong place), but for showing you the shape of a word it is excellent, and it already knows the rules of all five of Inkhaven’s languages.

TERM Hyphenation dictionary

A per-language set of rules (originally Liang’s algorithm, as used in TeX) for finding the points where a word may be broken across lines. Inkhaven borrows Typst’s bundled dictionaries — English, Russian, German, French, Spanish — to place syllable boundaries. Because the rules are conservative, the **cuts** can under-count; the **count** is done separately, below.

Where the count comes from

Here is a subtlety that trips up naïve tools: the number of syllable **cuts** is not always the number of **syllables**. Hyphenation is cautious and sometimes declines to break a word it nonetheless knows has two syllables. So Inkhaven does not count cuts — it counts **vowel nuclei**, the syllabic peaks, because every syllable has exactly one. “Fire” may resist hyphenation, but its vowels tell the true story.

Two language-specific rules ride on top:

- **English silent final-e.** English spells syllables it does not say. "Extensive" ends in a written **e** that is no syllable at all; "make" is one syllable, not two. Inkhaven drops the mute final **e** before counting, so English counts come out right where a pure vowel-counter would inflate them.
- **Every language keeps its own vowel set.** What counts as a nucleus differs by language — Russian's vowels are not English's, and **y** is a vowel in some positions and not others. The counter consults the right set for the declared language.

DOES IT WORK IN RUSSIAN?

It works **better** in Russian. English orthography hides its sounds — silent letters, the same spelling for different vowels — so English syllable counts are the roughest of the five. Russian spelling is close to phonemic: a written vowel is a spoken vowel. Ask for poetry syllabify --language ru стихотворение and the answer — сти·хот·'во·ре·ние (5 syl) — is exact, because there is nothing hidden to guess at.

Finding the stress

A syllable count with no stress is half a measurement — you know how **long** the word is but not its **shape**, and metre is all shape. Inkhaven marks the stressed syllable with **'**, and it finds it by a different route in each language, because stress lives in a different place in each:

Language	Default stress rule
English	the penultimate syllable (a rough default — English stress is lexical and truly unpredictable from spelling)
Russian	penult in two-syllable words, antepenult in longer ones — a heuristic, since Russian stress is famously mobile and unmarked
German	the initial syllable — the Germanic default, right far more often than not
Spanish	penult, unless the spelling (a final consonant, or a written accent) moves it
French	phrase-final — French stress is not a property of the word but of the phrase, falling on its last full syllable

These are **defaults**, and defaults are guesses. The honest ones — English and Russian — are marked as heuristics above, because no rule reads lexical stress off spelling alone without a pronouncing dictionary, which Inkhaven does not ship. This is the roughest edge in the whole poetry layer, and the book will not pretend otherwise. Which is exactly why the next feature exists.

Overriding stress with an accent mark

When the default is wrong — and in English or Russian it sometimes will be — you can tell Inkhaven the truth by writing an acute accent over the stressed vowel in your text: `présent` for the verb, `présent` for the noun. The engine sees the mark and takes it as gospel, overriding whatever the default rule would have guessed. In Russian verse, where poets and editors already mark stress on ambiguous words for exactly this reason, the convention is native — write `сто́йт` and the stress is fixed. The mark is for you and for the tool at once: it disambiguates the poem **and** corrects the scan.

THE ONE HONEST LIMITATION, STATED PLAINLY

Read from spelling alone, English stress is a guess and English syllable counts are approximate. Russian, German, and Spanish are far more reliable; French, being phrase-timed, sidesteps the problem. Where you need English metre to be exact — a tight scansion argument, a syllabic form — mark your stresses with accents and count by hand at the edges. A phoneme-dictionary backend that would fix this for English is a known, deferred improvement; until it lands, the accent mark is your recourse.

Scanning a whole line

Feed `syllabify` a line rather than a word with `--line`, and it breaks each word in turn — the raw material every later chapter builds on:

```
$ inkhaven poetry syllabify --line "Shall I compare thee to a
summer's day"
```

Shall	'Shall (1 syl)
I	'I (1 syl)
compare	'com·pare (2 syl)
thee	'thee (1 syl)
to	'to (1 syl)
a	'a (1 syl)
summer's	'sum·mer's (2 syl)
day	'day (1 syl)

Ten syllables, as a line of iambic pentameter should have. Two things to notice, both honest. Every word carries a stress mark, monosyllables included — the standalone syllabifier reports each word's **default** stress in isolation, with nothing to lean on but the language rule, so "compare" comes back `'com·pare` (the English penult default), though in context the stress is really on "-pare." And that is the deeper point: counting is not yet scanning. Knowing there are ten syllables does not tell you they alternate weak-strong in five neat feet, and the isolated per-word stress is only a first guess. The scanner of the next chapter reads stress **in context**, and gets "compare" right — which is where the poem starts to answer back.

WHAT YOU LEARNED

- Syllabification underlies everything: metre, rhyme, and syllabic forms all stand on it. `poetry syllabify` is the tool.
- Cuts come from Typst's bundled *hypher* hyphenation dictionaries (no new dependency); the **count** comes separately from vowel nuclei, so a cautious hyphenator's missed break never costs a syllable.
- English drops its silent final-e before counting; each language uses its own vowel set. Russian counts are near-exact because its spelling is near-phonemic; English is the roughest.
- Stress is found per language — penult, antepenult, initial, phrase-final — but English and Russian defaults are honest **heuristics**, not lexical truth.
- Write an acute accent over a vowel to override the guess and fix the stress — native convention in Russian verse, and your recourse in English until a phoneme dictionary lands.

PART II

Measure

CHAPTER 3

3

Metre and scansion

Scansion is the act of marking a line's rhythm — showing which syllables are stressed and which are not, and reading a pattern off the result. It is the oldest analytic tool in poetry, older than the printed page, and it is where a line stops being words and becomes **measure**. This chapter is about `poetry metre`, the command that scans a line, names the foot it hears, and — if you have declared a form — tells you how well the line keeps the promise the form made.

The vocabulary of the foot

A *foot* is the small repeating unit of a metre: a fixed little pattern of stressed and unstressed syllables that the line stacks end to end. English and German verse is *accentual-syllabic* — it counts both the syllables and the stresses — and its feet have names two and a half thousand years old:

Foot	Pattern	Example
iamb	unstressed–stressed	<i>be·low</i> — the heartbeat of English verse
trochee	stressed–unstressed	<i>gar·den</i> — falling, insistent
anapaest	unstressed–unstressed–stressed	<i>in· ter· vene</i> — galloping, the limerick’s foot
dactyl	stressed–unstressed–unstressed	<i>mer·ri·ly</i> — falling triple time

Stack five iambs and you have *iambic pentameter*, the ten-syllable line of Shakespeare and Milton — *penta* (five) *meter* (measure). Four of them is tetrameter, the ballad and Pushkin’s line; three, trimeter. The metre of a form is a foot and a count: **iambic** × 5.

TERM Foot

The repeating rhythmic unit of a metre — iamb, trochee, anapaest, dactyl — each a fixed pattern of stressed and unstressed syllables. The number of feet in a line gives the line its length-name: pentameter (5), tetrameter (4), trimeter (3).

Scanning a line

Hand **poetry metre** a clean line and it scans it. We start in Russian, with the opening of *Eugene Onegin* — because, as Chapter 2 warned, Russian is where the scanner is at its sharpest:

```
$ inkhaven poetry metre --line "Мой дядя самых честных правил" --
language ru
Мой дядя самых честных правил
· / × / × / × / ×   (9 syllables)
→ detected: iambic tetrameter (fit 1.00)
```

The middle row is the scan, and it uses **three** marks, not two:

Mark	Name	Meaning
/	stressed	a syllable the scanner is confident carries stress
×	unstressed	a syllable it is confident does not
·	flexible	a monosyllable, or a syllable whose stress depends on context — the scanner declines to fix it alone

Read the Russian line: **Мой** is a monosyllable (·, flexible), and then the four polysyllables — **дядя**, **самых**, **честных**, **правил** — each lay down a clean strong-weak pair (/ ×). The result is a textbook iambic tetrameter, and Inkhaven names it with a **conformance** of 1.00.

Conformance is the whole idea

That number is the soul of the poetry layer, so look at it closely. Conformance is **how closely the line matches the pattern** — 1.00 is a perfect fit; lower numbers mean the rhythm departs from the ideal. And departure is not error. The best lines are rarely the ones that score 1.00; a great poet leans on the metre rather than marching to it, and a lower fit may be exactly the felicity that makes the line live.

Inkhaven reports the number and says **nothing** about whether it is good. A 0.75 might be a masterstroke of variation or a line that has lost its footing; the tool cannot tell, and does not try. It measures the distance from the ideal; judging that distance is the poet's and the critic's work, never the machine's.

THE METRE ENGINE IS SHARED, AND BATTLE-TESTED

The scansion here is not new code written for poetry. It is the same metre-detection engine Inkhaven's ConLang suite uses to analyse the prosody of invented languages, pointed now at real verse. Rhythm is rhythm whether the language was born or built — reusing the engine means the poetry layer inherited a mature, well-exercised scanner the day it shipped, rather than a fresh one full of fresh bugs.

Why English fights back — and the accent mark

Now the honest part. Give the scanner the most famous line of iambic pentameter in English, unmarked, and watch it stumble:

```
$ inkhaven poetry metre --line "Shall I compare thee to a
summer's day"
  Shall I compare thee to a summer's day
  . . / x . . . / x .   (10 syllables)
  → detected: amphibrachic trimeter (fit 0.75)
```

Ten syllables, correctly counted — but a row full of `.`, and a nonsense verdict of "amphibrachic trimeter." What went wrong? Nothing, in fact: the scanner is being **principled**. Six of the line's eight words are monosyllables — *Shall, I, thee, to, a, day* — and a monosyllable's stress is not a property of the word but of the sentence around it. The scanner refuses to invent stresses it cannot justify, so it marks those syllables flexible and finds no firm pattern to lock onto. This is the same limitation Chapter 2 named: read from spelling alone, with no pronouncing dictionary and no syntax, English rhythm is genuinely underdetermined.

And here is the recourse, the same one Chapter 2 offered: **mark the stresses you mean**. Put an acute accent on the syllables that carry the beat, and the flexible marks resolve:

```
$ inkhaven poetry metre --line "Shall I compáre thee to a súmmers
day"
  Shall I compáre thee to a súmmers day
  . . x / . . . / x .   (10 syllables)
  → detected: iambic pentameter (fit 1.00)
```

Two accents — on *compáre* and *súmmers* — were enough to give the scanner its anchors, and it now hears the pentameter perfectly, fit 1.00. The lesson is the book's recurring one: in Russian the machinery just works; in English you sometimes have to tell the machine what your ear already knows, and the accent mark is how you say it.

Scanning against a declared form

Detection is what the critic wants — hand it an unknown line, hear its metre named. Checking is what the poet wants — **I said this would be iambic pentameter; is it?** Add `--form` and `poetry metre` measures the line against the form's declared metre rather than against whatever pattern it happens to resemble:

```
$ inkhaven poetry metre --line "Shall I compáre thee to a súmmers
day" --form sonnet
  Shall I compáre thee to a súmmers day
  · · × / · · · / × ·   (10 syllables)
  → detected: iambic pentameter (fit 1.00)
  → declared iambic (5 feet): 10 of 10 syllables, fit 1.00
```

The last line is the judgement against the **promise**: ten syllables of the ten the form asks for, a fit of 1.00 against the declared iamb. Feed the same command the **unmarked** line and that final fit drops to 0.50 — not because the poetry changed but because the evidence did. If you write an eleven-syllable line, Inkhaven reports it too — and may tag it, which brings us to the two most useful marks the scanner adds.

Feminine endings and catalexis

Two departures are so common, and so meaningful, that the scanner names them:

- A **feminine ending** is an extra unstressed syllable hanging off the end of a line — an eleventh syllable on a pentameter line. "To be or not to be, that is the question" ends on the unstressed `-tion`: a feminine ending. It is not a mistake; in Russian verse it is structural, alternating with masculine endings by rule. Inkhaven tags it `· feminine ending` rather than docking the line for being long.
- A **catalectic** line is one syllable **short** — a foot with its last syllable clipped off, a favourite trochaic trick for ending a line on a strong beat. Inkhaven tags it `· catalectic (one short)`.

Both tags are the tool distinguishing **deliberate metrical shapes** from **ragged counting**. An eleven-syllable line is not simply "wrong by one"; it may be a feminine ending doing exactly its job. Naming the shape is more useful — and more honest — than a bare error.

TERM Feminine ending

A line ending on an extra unstressed syllable, beyond the metre's nominal count (e.g. an eleventh syllable on an iambic-pentameter line). Its opposite, ending on a stress, is a *masculine ending*. In Russian verse the two alternate by convention; in English they are a source of variation.

The three traditions of measure

Not every poetry counts the way English does, and a tool that only knew accentual-syllabic verse would mismeasure most of the world's poetry. The `metre_tradition` field in a poem block tells Inkhaven which system to apply:

- **Accentual-syllabic** (English, German) — count both syllables and stresses. The default, and everything above.
- **Syllabic** (French, and the Japanese forms) — count **syllables only**; stress is not metrical. A French alexandrine is twelve syllables, full stop; a haiku is 5–7–5 syllables and cares nothing for which are stressed. Inkhaven measures the count and ignores the stress, as the tradition demands.
- **Accentual** (Old English, much folk and rap verse) — count **stresses only**; the number of unstressed syllables between them is free. A four-beat line is four strong beats however many weak ones it carries.

Declaring the tradition is not a formality — it changes what "keeping the metre" **means**. Score a French alexandrine as if it were accentual-syllabic and you will find fault where there is none, because you are measuring a thing the tradition never promised. The multilingual promise of the workbench runs all the way down to this: each language's verse is measured by its own tradition's rules, not English's.

FREE VERSE IS MEASURED TOO, NOT EXCUSED

Verse with no regular metre is not beyond measurement — it is measured **differently**. Inkhaven can profile free verse by its rhythm of line lengths and stress density, the way a critic describes the cadence of Whitman or a psalm without scanning feet. Absence of metre is a positive style, and the workbench reports its shape rather than shrugging.

WHAT YOU LEARNED

- A *foot* is a metre's repeating unit — iamb, trochee, anapaest, dactyl; its count names the line — pentameter, tetrameter, trimeter.
- `poetry metre` scans a line into three marks — / stressed, × unstressed, · flexible (a monosyllable or context-dependent syllable) — names the metre, and reports a *conformance*, the distance from the metrical ideal. Departure is variation, not error; the tool measures it and judges it not.
- In Russian the scan is sharp (Onegin's line reads iambic tetrameter, fit 1.00, unmarked). English monosyllabic lines read "irregular" because the scanner **principledly** won't invent stresses — mark the beats with an acute accent and the pattern resolves.
- Add `--form` to check a line against a **declared** metre rather than a guessed one — the poet's question, not the critic's.
- The scanner names *feminine endings* (an extra unstressed syllable) and *catalectic* lines (one short), distinguishing deliberate metrical shapes from ragged counting.
- Three traditions — accentual-syllabic, syllabic, accentual — measure different things; the `metre_tradition` field picks the right one, so French and Japanese verse are measured by their own rules. Even free verse gets a profile.

CHAPTER 4

4

Rhyme

Rhyme looks simple — two words that end alike — and is not. Whether two words rhyme depends on where their stress falls, on sounds the spelling hides, and on which language's ear is listening: a pair that rhymes richly in Russian may barely chime in English, and a pair the eye accepts the ear rejects. **poetry rhyme** is the tool that takes two words and tells you not just **whether** they rhyme but **how** — its quality, its type, and the tail they share.

Asking whether two words rhyme

\$ inkhaven poetry rhyme day may
day / may: perfect masculine rhyme on "-ay"

Three judgements again, as with syllables. The rhyme's **quality** is perfect; its **type** is masculine; and the **shared tail** — the stretch of sound the two words hold in common — is **-ay**. When two words do not rhyme, it says so plainly:

\$ inkhaven poetry rhyme day tree
day / tree: no rhyme

The four qualities of rhyme

Inkhaven grades a rhyme on a four-step scale, from a true chime down to a mere agreement of spelling:

Quality	What it means
Perfect	the sounds from the stressed vowel onward agree completely — <i>day / may, light / night</i> . The rhyme the ear wants.
Near	the tails are close but not identical — a slant or half rhyme, <i>bind / mind, shape / keep</i> . The workhorse of modern verse.
Eye	the words look like they should rhyme but do not sound alike — <i>love / move, cough / bough</i> . A rhyme for the eye, betrayed by the ear.
None	no meaningful agreement of sound or spelling.

The distinction between **near** and **eye** is the subtle one, and it is where a tool reading plain text reaches its limit. A near rhyme is close in **sound**; an eye rhyme is close only in **spelling**. Telling them apart requires knowing how the words are pronounced — which, in English, spelling does not reliably reveal. Watch the limit bite:

\$ inkhaven poetry rhyme love move
love / move: perfect masculine rhyme on "-ove"

The tool calls it **perfect** — because, reading the spelling, it sees the identical tail **-ove** and has no way to know that "love" and "move" part company in the mouth.

This is a textbook eye rhyme, and Inkhaven has confidently mis-graded it. The failure is not sloppiness; it is the honest consequence of judging sound from spelling in a language whose spelling lies.

THE ENGLISH EYE-RHYME PROBLEM, STATED HONESTLY

In a language whose spelling matched its sound, eye rhyme would not exist as a separate category — if it looks alike it sounds alike. English is not that language. "Love" and "move" share four letters and rhyme for the eye alone; "cough," "bough," "through," and "though" all end in -ough and no two of them rhyme. Read from spelling with no pronouncing dictionary, Inkhaven **cannot reliably tell a perfect or near rhyme from an eye rhyme in English** — as the love / move verdict above shows, it will sometimes stamp an eye rhyme "perfect." This is the single hardest limit in the rhyme engine, and it is an English limit specifically. Where it matters to your argument, trust your own ear over the tool's.

Masculine, feminine, dactylic

A rhyme's **type** is about where the stress sits relative to the rhyming sound — how many syllables the chime spans:

- **Masculine** — the rhyme falls on the final, stressed syllable: *be*low* / *a*glow*. Single, hard, closing.
- **Feminine** — a stressed syllable followed by an unstressed one, both rhyming: *moth·er* / *broth·er*. Softer, a falling close.
- **Dactylic** — the rhyme spans three syllables, stressed–unstressed–unstressed: *quiv·er·ing* / *shiv·er·ing*. Rare and virtuosic in English; ordinary in Russian.

The type is not decoration — it is structural. Russian verse alternates masculine and feminine rhymes by strict convention, and a form's rhyme scheme often implies its rhyme types. Naming the type lets the completion tools of Chapter 6 check that a poem's rhymes fall in the pattern its tradition demands.

TERM Rhyme tail

The stretch of a word from its stressed vowel to its end — the part that must agree for a rhyme to hold. "Delight" and "tonight" share the tail **-ight**; the rhyme is judged on the tail, not the whole word.

Every language rhymes differently

This is the chapter where the multilingual promise does its most interesting work, because what counts as a rhyme is genuinely different in each language. Before comparing two tails, Inkhaven **normalises** them by the target language's own rules of sound:

- **German** devoices final consonants — a **d** at the end of a word is pronounced **t**, **b** becomes **p**, **g** becomes **k**. So German *Rad* and *Rat* rhyme perfectly to the ear though they end in different letters, and Inkhaven, devoicing before it compares, hears the rhyme the German ear hears.
- **French** drops the mute final **e**. *Aimée* and *aimer* agree once the silent ending is stripped. French also lets the eye rule more than most traditions — a legacy of its spelling — and Inkhaven's normalisation follows the convention.
- **Russian** reduces unstressed **o** toward **a** (the reduction called *akanye*), so words that differ on paper can chime in the mouth. Normalising for it lets Inkhaven grade Russian rhyme the way a Russian ear grades it — and Russian, with its rich inflectional endings, rhymes so easily that the tradition prizes the **unexpected** rhyme over the merely correct one.

```
$ inkhaven poetry rhyme Rad Rat --language de
Rad / Rat: perfect masculine rhyme on "-at"
```

Two differently-spelled words, and Inkhaven calls them a perfect rhyme — because it devoiced the final **d** to **t** before comparing, hearing the rhyme the German ear hears rather than the difference the eye sees. The normalisation is silent here, but it is doing real work: without it, **Rad** and **Rat** would look like a near rhyme at best. And in Russian the same machinery, running the *akanye* reduction, grades a clean feminine rhyme:

\$ inkhaven poetry rhyme ночи очи --language ru
 ночи / очи: perfect feminine rhyme on “-очи”

DOES IT WORK IN RUSSIAN? HERE, BEST OF ALL

Rhyme is the feature where Russian most outshines English. Russian spelling tracks sound closely, its stress (once marked) is unambiguous, and its rhyme tradition is precise and well-codified. Inkhaven’s Russian rhyme judgements are correspondingly sharp — the akanye normalisation, feminine/masculine alternation, and rich inflectional rhymes all fall out cleanly. One caution carried over from Chapter 2: Russian’s mobile stress still needs marking on longer words for the tool to place the tail correctly, so mark your stresses and the rhyme engine rewards you. If you want to **see** it at its most accurate, give it a marked line of Pushkin.

WHAT YOU LEARNED

- poetry rhyme grades two words on **quality** (perfect / near / eye / none), **type** (masculine / feminine / dactylic), and the **shared tail** they chime on.
- **Near** rhyme is close in sound; **eye** rhyme is close only in spelling. In English, read from spelling alone, the two cannot always be told apart — the rhyme engine’s hardest limit, and an English-specific one.
- Rhyme **type** is structural, not cosmetic: Russian verse alternates masculine and feminine rhymes by rule, and the completion tools check for it.
- Each language is **normalised** by its own rules before comparison — German final-consonant devoicing, French mute-e, Russian akanye — and the tool reports the normalisation so you can inspect its reasoning.
- Russian is where the rhyme engine is sharpest, because its spelling tracks its sound; English is where it is roughest.

POETRY WITH INKHAVEN

PART III

The Whole Poem

CHAPTER 5

5

The Inner Poet

The chapters so far handed you tools you point at one line at a time — scan this, rhyme that. This one introduces a **reader**: a standing companion who takes in a whole stanza at once and tells you how it sits against the form you declared. The Inner Poet is one of Inkhaven's family of "Inner" readers — the Inner Editor, the Inner Socrates, the Inner Theologian, and now the Inner Poet — each a second pair of eyes with a particular expertise, each bound by the same iron rule: **observe, measure, report; never rewrite.**

Two speeds of reading

The Inner Poet reads at two speeds, and the distinction runs through the whole family.

The **fast track** is mechanical and instant. It needs no network, no model, no waiting — it runs the scansion and rhyme engines of the last two chapters over every line of a stanza and reports what it measures. It is deterministic: the same stanza yields the same findings every time. This is the track you leave running as you write.

The **slow track** is reflective. It hands the poem to a language model with a carefully bounded brief and asks for observations a ruler cannot make — where the imagery strains, whether the volta turns, how the sound and the sense pull together or apart. It costs a call and a moment's wait, and you invoke it deliberately, when you want a considered second reading rather than a measurement.

The fast track: findings

On the command line the fast track is `poetry scan`; in the editor it is the Inner Poet's instant read. Give it a stanza and its declared form, and it returns a list of **findings**:

```
$ inkhaven poetry scan --form heroic_couplet --language en \
  --text "The trees are green in summer and in spring
          The autumn leaves will fall upon the ground"
♪ Praise    [Metre]   Line 1 scans cleanly as iambic pentameter.
♪ Praise    [Metre]   Line 2 scans cleanly as iambic pentameter.
♪ Concern   [Rhyme]   Lines 1↔2 (A-A): "spring" / "ground" — do not
rhyme.
```

Three findings, and they show the whole grammar of the fast track. Each carries a **severity**, a **kind** (Metre or Rhyme), and a plain-language **message** tied to a line. The couplet's two lines each keep their pentameter — earning Praise — but the form's rhyme scheme said lines 1 and 2 must chime (A-A), and "spring" and "ground" do not. That is a Concern, and it names exactly which promise was broken and where.

The three severities

The Inner Poet grades every finding on a three-step scale, and the scale is the whole philosophy in miniature:

Severity	What it says — and does not say
Praise	a line meets its declared target cleanly. The tool notices what works , not only what fails — a reader that only ever complains is soon ignored.
Note	a line departs from the target in a way worth seeing — a fit of 67%, a line two syllables short. Not an error: a note is an observation you may have meant, or may not.
Concern	a promise is plainly broken — a declared rhyme that does not rhyme, a form line count that cannot be right. The strongest word the tool will use, and still only a flag, never a fix.

Notice what is absent: there is no "Error," no red, no imperative. The harshest thing the Inner Poet will say is **Concern**, and even a Concern is a raised eyebrow, not a command. Whether "spring" and "ground" ought to rhyme is your call — perhaps you are writing a deliberately unrhymed couplet and the Concern is noise; perhaps you forgot the scheme and it just saved you. The tool cannot know which, so it flags and stops.

PRAISE IS A FEATURE, NOT POLITENESS

It would have been easy to build a tool that only reports problems. The Inner Poet reports successes too, and that is deliberate. A line that scans cleanly is **information** — it tells you the metre is holding, so that when a Note appears three lines later you know it is a real departure and not the tool finally waking up. Praise calibrates the silence around the warnings.

Running the fast track in the editor

Inside Inkhaven, the Inner Poet lives with the rest of the family under the Inner-reader menu. Open it with **Ctrl+B J**, then press **P** for the Poet. On the poem paragraph under your cursor, press:

- **F** — **fast check**. Runs the scan you just saw over the current stanza and posts its findings to the Output pane, each tied to its line, instantly and offline.

- **E** — **engage**. Starts the slow track (below).

The findings land in the Output pane under the **J** mark, filterable like every other reader's output, and they persist until you act on them — so a Concern does not scroll away unaddressed. Nothing is written to your poem; the findings are notes **about** the text, set beside it, never in it.

The slow track: engaging the Poet

Press **E**, and the Inner Poet reads reflectively. It sends the stanza — with its declared form and the fast track's own findings for context — to a language model under a strict system brief, and waits for a considered response. The brief is the constitution of the feature, and it is worth knowing what it forbids:

- The model is told, in its opening instruction, that it is an **observer** — that it must describe and question, never rewrite, never hand back "improved" lines.
- It is asked for observations a ruler cannot make: whether an image earns its place, whether the argument of a sonnet actually turns at its volta, where sound reinforces sense and where it works against it.
- It answers in the poem's own language. Engage it on a Russian stanza and the observations come back in Russian, keyed to Russian prosody — the multilingual promise holds here as everywhere.

The response arrives as a **thought** in the Inner Poet's panel — prose for you to read and weigh, headed **J Inner Poet**. It is advisory in the fullest sense: not a diff, not a suggestion you accept or reject with a keystroke, but a reading you consider. The poem on the page is untouched, and stays untouched, unless **you** change it.

THE LINE THE INNER POET WILL NOT CROSS

This is the same principle that governs every AI feature in Inkhaven, and it is absolute: **the AI never edits your prose or your verse**. The slow track produces words about your poem, never a rewrite of it. There is no "apply" button, because there is nothing to apply — the Poet gives you a reading, and what you do with a reading is the one thing a workbench must always leave to the writer. If you want the poem changed, you change it, and then you can ask the Poet to read it again.

WHAT YOU LEARNED

- The Inner Poet is one of Inkhaven's "Inner" readers — a standing companion that takes in a whole stanza and reports how it sits against the declared form. Same iron rule as the family: observe and report, never rewrite.
- It reads at two speeds: a **fast track** (deterministic, offline scansion + rhyme, run with `poetry scan` or `Ctrl+B J → P → F`) and a **slow track** (a bounded LLM reading, run with `E`).
- Fast-track **findings** carry a severity — **Praise** (a target met), **Note** (a departure worth seeing, not an error), **Concern** (a promise plainly broken). The harshest word is Concern, and even that is a flag, not a fix.
- Praise is deliberate: reporting what works calibrates the weight of the warnings.
- The slow track sends the poem to a model under a strict observer's brief, answers in the poem's own language, and returns a **reading** — never a rewrite. No AI feature in Inkhaven ever edits your verse.

CHAPTER 6

6

Form and completion

A sonnet is not fourteen good lines; it is fourteen lines **in an order**, with a turn near the end and a rhyme scheme that binds them. A villanelle is a machine of repetition — two lines that come back, on a schedule, until the last stanza gathers them both. These forms have architecture, and architecture can be half-built. This

chapter is about **poetry status**: the tool that tells you how much of a form's structure you have raised, and whether what you have raised is sound.

How far along are you?

Where the last chapter measured lines, this one measures **the whole poem against its plan**. Ask for a poem's status and Inkhaven counts what you have written against what the form requires:

```
$ inkhaven poetry status --form villanelle --language en \
  --text "Do not go gentle into that good night
         Old age should burn and rave at close of day
         Rage rage against the dying of the light"
J villanelle · 3/19 · drafting
no structural issues
```

Three lines of the nineteen a villanelle demands: **3/19**, and the state is **drafting**. When the architecture is complete, the state flips:

```
$ inkhaven poetry status --form haiku --language ru \
  --text "Тихий старый пруд
         В воду прыгнула лягушка
         Всплеск в тишине"
J haiku · 3/3 · complete
no structural issues
```

A Russian haiku, three lines of three, **complete** — the completion check counts syllabic architecture in Russian exactly as it does in English, because a haiku is a haiku in any language.

What 'expected' means

The line target is not one rule but several, because forms count differently:

- The **fixed forms** carry their number in their bones — a sonnet is 14, a villanelle 19, a haiku 3, a tanka and a limerick 5. Inkhaven knows these outright.
- The **stanzaic forms** compute it — a form declared as six stanzas of three lines expects eighteen, and the target moves with the **stanzas** and **lines_per_stanza** you declared.
- The **open forms** — ode, elegy, free verse — have no fixed length, and **status** says so, reporting the line count as information rather than measuring it against a target that does not exist.

TERM Completion

The state of a poem relative to its form's required architecture: *drafting* while lines are still missing, *complete* once the full structure is present. It is a structural count, not a quality judgement — a complete poem is finished in **shape**, which is the only thing a machine can vouch for.

Structural issues

Counting lines is the easy half. The interesting half is checking that the lines you have obey the form's internal **rules** — and the fixed forms have rules that go well beyond their length. When **status** finds one broken, it lists it under the count rather than saying "no structural issues":

- A **villanelle** runs on two refrains: line 1 and line 3 of the first tercet return, in strict alternation, as the last line of each following stanza, and both close the final quatrain. **status** checks that the refrains you owe have actually come back where the schedule places them.
- A **pantoum** interlocks: lines 2 and 4 of each quatrain become lines 1 and 3 of the next. The check follows the chain and flags a link that does not hold.
- A **ghazal** is built on a *radif* — a word or phrase repeated at the end of both lines of the opening couplet and the second line of every couplet after — often with a **signature** (the poet's name) in the closing couplet, the *maqta*. **status** looks for the radif on its schedule.
- A **sonnet** turns. Somewhere — classically at line 9 in the Italian, near line 13 in the English — the argument pivots: the *volta*. The sonnet check looks for the turn the form promises.

TERM Volta

The "turn" of a sonnet — the point where its argument shifts: a problem giving way to a resolution, a question to an answer, a scene to its meaning. In the Petrarchan sonnet it falls between octave and sestet (line 9); in the Shakespearean it often waits for the closing couplet. A sonnet without a volta is fourteen lines that never change their mind.

These checks are where `status` earns its place beside `scan`. The fast track of Chapter 5 reads line by line and would never notice that a villanelle's refrain failed to return — that is a fact about the **whole poem's shape**, invisible from any single line. `status` sees the architecture.

DRAFTING IS A FIRST-CLASS STATE, NOT A FAILURE

A poem at 3/19 is not broken — it is **in progress**, and Inkhaven treats that as a normal, honourable condition rather than a pile of missing-line errors. This is why a form can carry `suppress_while_drafting`: so the Inner Poet holds its structural complaints until the poem is complete enough for them to be fair. Measuring an unfinished poem against a finished poem's rules would be pedantry, and the workbench declines to be pedantic. It waits.

Status in the workflow

`status` is the bird's-eye view you check between sessions: **where is this poem, and is its scaffolding sound?** In the editor the same completion count feeds the outline — a poem's entry can show how far along it is at a glance — so a book of villanelles tells you which are finished and which still owe refrains without your opening a single one. On the command line, `status --json` (Chapter 8) turns the same reckoning into data a script can gate a build on: a manuscript that reports every poem `complete`, or a CI job that will not pass until they are.

WHAT YOU LEARNED

- `poetry status` measures the **whole poem against its form's plan**: a line count (`written / expected`) and a state — **drafting** or **complete**.
- The expected count comes three ways: fixed in the form (sonnet 14, villanelle 19, haiku 3), computed from declared `stanzas × lines_per_stanza`, or absent for open forms, which are counted but not judged.
- Beyond counting, `status` checks each fixed form's internal rules — a villanelle's refrain schedule, a pantoum's interlock, a ghazal's radif, a sonnet's volta — and lists any that are broken. These are whole-poem facts the line-by-line fast track cannot see.
- **Drafting** is a first-class state; `suppress_while_drafting` lets the Poet hold structural complaints until a poem is complete enough for them to be fair.
- The completion count feeds the outline and, via `status --json`, feeds scripts and CI.

PART IV

Beyond the Original

CHAPTER 7

7

Translating verse

To translate a poem is to face an impossibility and choose how to fail. A line of verse carries three things at once — its **form** (the metre and rhyme), its **meaning** (what the words say), and its **sound** (how they ring, alliterate, and chime) — and no translation into another language keeps all three intact. Preserve the metre and you bend the sense; chase the exact meaning and the music is gone; reproduce the sound and you have left the dictionary behind. This is the translator's **trilemma**, and Inkhaven's contribution is not to solve it — nothing can — but to **measure the trade** you actually made.

The verse-translation paragraph

Translation is where the sixth member of the verse family, `para:verse-translation` ($\rightleftharpoons$), earns its place. It holds a translated stanza **paired with its source** — the original and its rendering kept together as one structural unit, so the two never drift apart in the manuscript and every tool that reads the paragraph can see both sides at once. A book of translations built this way is a parallel text by construction: the outline's $\rightleftharpoons$ marks show you, at a glance, which stanzas are original and which carry a source across.

The trilemma, scored

`poetry trilemma` takes a source line or stanza and its translation and reports how the translation fares on each axis of the impossibility:

```
$ inkhaven poetry trilemma --form sonnet --language ru --to-
language en \
  --source "Я вас любил: любовь ещё, быть может" \
  --translation "I loved you once, and still perhaps I do"
Translation trilemma (ru → en):
```

Form		50%	0/1 lines keep the foot · the source does not rhyme...
Meaning			(the AI axis — engage the Inner Poet in the editor)
Sound		83%	alliteration density 0.17 → 0.00

Three bars, one axis apiece. Two of them the workbench fills in mechanically; the third it pointedly leaves empty, and that emptiness is the most honest thing on the readout.

Form — measured

The **form** score asks how well the translation keeps the original's shape: does it hold the declared metre, foot for foot, and does it rhyme where the scheme says it should? It is computed from the same scansion and rhyme engines you already know, run on both sides and compared. A translation that renders a Russian iambic pentameter as an English iambic pentameter, rhyming where the sonnet rhymes,

scores high; one that keeps the words and drops the metre scores low. Give the tool two clean, marked pentameter lines and the foot axis reads 100%:

Form  100% 1/1 lines keep the foot · ...

The note beside the bar shows its working — how many lines kept the foot, and what it found of the rhyme — so a bare percentage never has to be taken on trust.

Sound — measured, roughly

The **sound** score asks whether the translation carries the original's **texture** across — its density of alliteration and repeated sound, the sonic weave beneath the words. Inkhaven measures this by comparing sound-repetition profiles on the two sides: the readout above reports the source's alliteration density falling from 0.17 to 0.00 in the translation, a concrete sign that the English has smoothed out music the Russian carried. This is the roughest of the three measures — sound similarity across languages with different phoneme inventories is genuinely hard, and the tool measures a proxy, not the thing itself — but a proxy you can see beats an impression you cannot.

Meaning — left to judgement

The **meaning** axis is blank, always, and deliberately. Whether a translation is **faithful to the sense** — whether "still perhaps I do" carries what Pushkin's line means, with its weight and its reticence — is not a thing a ruler measures. It requires reading, and taste, and knowledge of both languages' connotations. So the trilemma refuses to fake a number here and instead points you to the one part of the workbench equipped for the question: the **Inner Poet's slow track** (Chapter 5). Engage it on a verse-translation paragraph and it reads both sides and offers observations on the meaning carried and the meaning lost — as prose to weigh, never a score to trust.

TWO AXES A MACHINE CAN HOLD, ONE IT MUST NOT

The trilemma readout is a small manifesto for the whole poetry layer. Form and sound are **countable**, so Inkhaven counts them and shows its working. Meaning is a matter of **judgement**, so Inkhaven leaves it to a reader — you, or the Inner Poet reading for you — and marks the axis empty rather than inventing a figure. A tool that scored "meaning fidelity: 71%" would be lying with a decimal point. The honest readout has a blank where the judgement goes.

Reading a translation as a critic

The trilemma is as much the critic's tool as the translator's. Point it at an existing translation — Nabokov's deliberately literal English *Eugene Onegin* against a rhyming verse translation of the same Pushkin, or two Englishings of Rilke — and it quantifies, in a form you can put in an argument, the choice each translator made: this one kept the form and paid in sound; that one preserved the music and loosened the metre. The numbers do not settle which is **better** — that judgement is yours, as always — but they make the trade-off visible and citable, which is exactly what a close reading of a translation needs and rarely has.

WHAT YOU LEARNED

- Verse translation faces a **trilemma**: form, meaning, and sound cannot all survive the crossing. Inkhaven does not solve it — it measures the trade you made.
- **para:verse-translation** ($\Leftrightarrow$) pairs a translated stanza with its source as one unit, so a book of translations is a parallel text by construction.
- **poetry trilemma** scores **form** (metre + rhyme fidelity, from the familiar engines) and **sound** (alliteration/repetition texture carried across — the roughest measure), each with its working shown.
- The **meaning** axis is left deliberately blank: semantic fidelity is judgement, not measurement, so the tool points to the Inner Poet's slow track rather than faking a number.
- The same tool serves the critic: it makes a translator's trade-offs visible and citable, without presuming to say which trade was right.

CHAPTER 8

8

Scripting and scholarship

Every chapter so far measured one poem at a time. But a scholar rarely has one poem — she has an anthology, a poet’s collected works, a corpus of a hundred sonnets she wants to characterise all at once. And a working poet assembling a manuscript wants a gate: a check that every poem in the book is finished before it goes to the printer. Both needs are the same need — the measurements of this book, applied in bulk and read by a machine. This final chapter is about the two doors Inkhaven opens for that: structured output on the command line, and the poetry engines exposed to Bund, Inkhaven’s scripting language.

Machine-readable output: --json

Every reporting command in this book has a `--json` flag, and it turns a human readout into data a program can consume. Where `poetry scan` prints ♪ marks for a person to read, `poetry scan --json` prints a structure for a script to parse:

```
$ inkhaven poetry scan --form heroic_couplet --language en --json \
\
  --text "The trees are green in summer and in spring
        The autumn leaves will fall upon the ground"
{
  "form": "heroic_couplet",
  "language": "en",
  "concerns": 1,
  "findings": [
    { "severity": "praise", "kind": "Metre", "line": 1,
      "message": "Line 1 scans cleanly as iambic pentameter." },
    { "severity": "praise", "kind": "Metre", "line": 2,
      "message": "Line 2 scans cleanly as iambic pentameter." },
    { "severity": "concern", "kind": "Rhyme", "line": 2,
      "message": "Lines 1↔2 (A-A): "spring" / "ground" – do not
rhyme." }
  ]
}
```

The same findings you saw in Chapter 5, now with a `concerns` count at the top and every finding a record with `severity`, `kind`, `line`, and `message`. `poetry status --json` and `poetry rhyme --json` do the same for their reports. Pipe any of them through a JSON tool and you can tabulate the metres of an anthology, count the rhyme types across a poet's career, or list every poem in a manuscript that is still `drafting` — the close reading of Chapters 3 and 4, run over a library instead of a line.

A gate for the manuscript: --fail-on-concern

One flag turns measurement into enforcement. `poetry scan --fail-on-concern` exits with a non-zero status the moment a stanza raises a Concern:

```
$ inkhaven poetry scan --form heroic_couplet --fail-on-concern \
  --text "The trees are green in summer and in spring
        The autumn leaves will fall upon the ground"
♪ Praise    [Metre]   Line 1 scans cleanly as iambic pentameter.
♪ Praise    [Metre]   Line 2 scans cleanly as iambic pentameter.
♪ Concern   [Rhyme]   Lines 1↔2 (A-A): "spring" / "ground" – do not
rhyme.
$ echo $?
1
```

poetry scan

That exit code is all a continuous-integration job needs. Put `--fail-on-concern` in the build of a book of formal verse, and the build fails if any poem has drifted from its declared form — a rhyme that stopped rhyming after an edit, a line that lost its metre in revision. The poet’s promise, checked automatically, on every commit. Note the restraint in the threshold: only a **Concern** fails the build, never a mere **Note**. A line two syllables short is your business; a declared rhyme that does not rhyme is a broken promise worth stopping for. The gate enforces only what the poet plainly meant.

THE PERMISSIVE PRINCIPLE, ONE LAST TIME

A build gate could have been a nag — failing on every Note, forbidding an off-metre line. It is not. `--fail-on-concern` is **opt-in** (the default scan never sets an exit code), and even when set it stops only for Concerns. Inkhaven measures freely and blocks rarely; the cost of a rough line is information, not obstruction. You choose to install the gate, and you choose how high to hang it.

The engines in Bund

Inkhaven’s scripting language, Bund, is a concatenative stack language for driving the editor from code, and the poetry engines are exposed to it as a family of `ink.poem.*` words (each also answering to the shorter `poem.*`). Where `--json` gives you the **commands**’ output, Bund gives you the **engines** directly, to compose into analyses the fixed commands do not offer. Try them with `inkhaven bund`:

```
$ inkhaven bund '"extensive" "en" ink.poem.syllable_count'
3

$ inkhaven bund '"day" "may" "en" ink.poem.rhyme'
{ "shared": "ay", "quality": "perfect", "type": "masculine" }

$ inkhaven bund '"Мой дядя самых честных правил" "ru"
poem.scan_line'
{ "pattern": "·/x/x/x/x", "syllables": 9, "metre": "iambic
tetrameter" }
```

Four words cover the layer's measurements: `syllable_count` (a word and its language → a count), `scan_line` (a line and its language → its scansion), `rhyme` (two words and a language → a rhyme analysis), and `status` (a text, a form, and a language → a completion report). Each takes its arguments off the stack and pushes back a plain value or a dictionary, ready to feed the next word. A Bund script can walk every paragraph of a book, scan the verse ones, and accumulate a table of metres — the whole anthology characterised in a loop, using the identical engines this book has described a command at a time.

Read-only by default, and by category

Every `ink.poem.*` word is classified `store_read` in Inkhaven's scripting policy — the permission category for words that **look at** the project without changing it. This is not a footnote; it is a guarantee. The poetry engines **cannot** alter your manuscript, because the category system will not let a read-word write. A script that mis-scans a thousand poems has done nothing but compute; your text is untouched, by construction, at the level of the permission model. It is the observe-never-write rule of the whole poetry layer, enforced not by good intentions but by the same machinery that guards every other read-only word in the language.

DOES IT WORK IN RUSSIAN? ALL THE WAY DOWN

The multilingual promise reaches even here. Every Bund word takes a language argument, and every one keys its behaviour off it — `poem.scan_line` reads a Russian line by Russian prosody, `poem.rhyme` grades a German pair by German devoicing. The scripting layer is not an English convenience with the other languages bolted on; it is the same five-language engine, addressable from code. A scholar of Russian verse scripts against it exactly as a scholar of English does.

Where the workbench ends

We have come the whole way — from a single syllable, through the scanned line and the graded rhyme, to the Inner Poet’s reading, the completeness of a form, the trilemma of translation, and now a whole corpus measured in a loop. At every step the tool did the same thing and refused the same thing: it counted, classified, and reported; it never wrote a word of your poem, never told you a line was good or bad, never crossed from measurement into composition. That refusal is not a missing feature. It is the feature — the reason the workbench can be trusted with your verse and with your argument about someone else’s. The ruler does not hold the pen. You do.

WHAT YOU LEARNED

- Every reporting command takes `--json` — structured output (findings with severity / kind / line / message, a `concerns` count, completion reports) for tabulating an anthology or auditing a manuscript in bulk.
- `poetry scan --fail-on-concern` gives a non-zero exit on any Concern — a CI gate that fails a build when a poem drifts from its declared form. Opt-in, and it stops only for Concerns, never Notes.
- The engines are exposed to Bund as `ink.poem.*` (short: `poem.*`) — `syllable_count`, `scan_line`, `rhyme`, `status` — composable words for analyses the fixed commands do not offer, over a whole book in a loop.
- Every poem word is `store_read` in the scripting policy: the permission model **forbids** it from altering your text — the observe-never-write rule enforced by machinery, not intention. And every word takes a language, so the scripting layer is fully multilingual.
- Across the whole workbench the discipline never broke: measure and report, never compose. The ruler does not hold the pen.

PART V

At the Desk

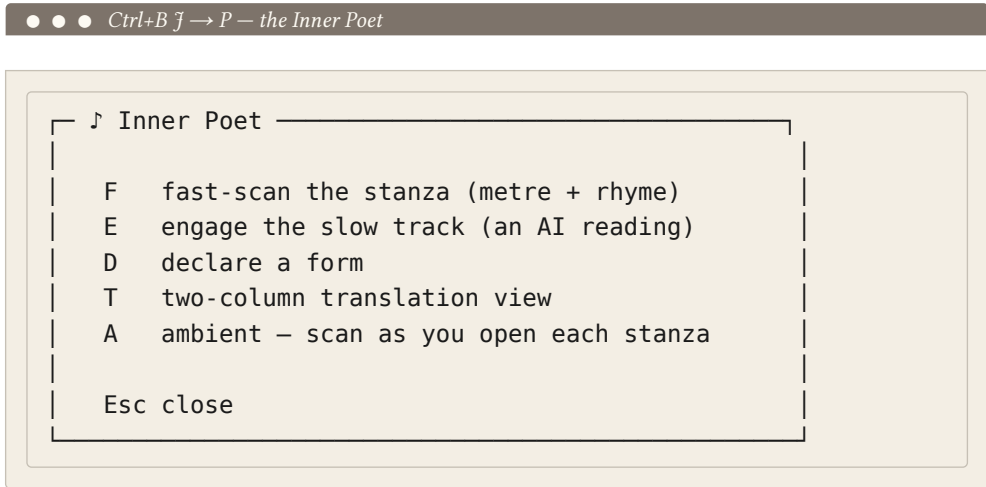
CHAPTER 9

9

Writing a poem at the desk

Every chapter so far measured verse from the command line — a line here, a stanza there, handed to `inkhaven poetry` and read back. That is the right way to **learn** the tools, and the right way to interrogate a poem that already exists. But it is not how you **write** one. Writing happens at the desk, in the editor, with the poem in front of you and the measuring running quietly alongside — and everything the earlier chapters did on the command line, the editor does without your ever leaving the page. This chapter is the whole loop, in one sitting: from an empty paragraph to a measured, form-checked poem.

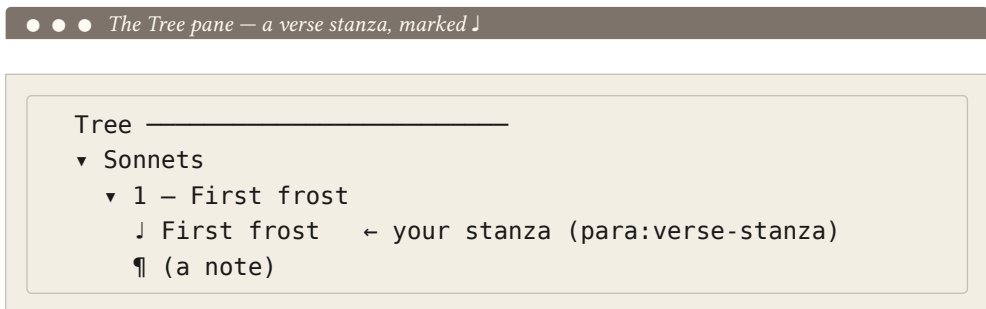
Everything lives under one door. Press **Ctrl+B J** to open the family of Inner readers, then **P** for the Poet:



Six keys, and the rest of this chapter is what they do.

Make a stanza

A poem is paragraphs of the verse family (Chapter 1). In the Tree pane, add a paragraph and give it a verse type — press **i** for the structural-type picker and choose **verse stanza**, or cycle an existing paragraph's type with **t** until the **J** appears. The glyph is your confirmation that Inkhaven now holds this block as verse, not prose:



Type your lines into it. Inside a verse paragraph the line breaks you type are the line breaks that print — Inkhaven never reflows verse.

Declare the form

A stanza on its own is just lines; what lets the Poet **measure** it is a declared form. From the Inner Poet (Ctrl+B J → P), press **D** for the form picker:

- ● ● $Ctrl+B \text{ } \mathcal{J} \rightarrow P \rightarrow D$ – the form picker

```

└─ ♪ Declare a form · D ───────────
|
|   > sonnet                      14 lines of iambic pentameter
|     petrarchan_sonnet          Italian: octave + sestet
|     villanelle                  19 lines, two refrains
|     haiku                       3 lines, 5-7-5 syllables
|     ...
|
| writes a poem: block beside the stanza – no verse
| is generated · ↑↓ select · Enter attach · Esc cancel

```

Choose a form and press Enter. Inkhaven writes its **poem:** block — localised to your project's language — as a small sidecar beside the stanza, exactly where the measuring tools look for it. This is the one act the command-line workflow could not do for you at the desk, and it is what turns the passive ruler into an active second reader. It writes a **form declaration**, never a line of verse; the poem stays entirely yours.

LOCALISED TO YOUR LANGUAGE

Declare a sonnet in a Russian project and the block comes back with the Russian iambic conventions set(`allow_pyrrhic`, `require_final_stress`); in an English one, the plain English scheme. The same picker, the right defaults – the multilingual promise, at the desk.

Write, and watch the count

Now write, and look at the status bar. While a verse paragraph is open, Inkhaven shows a live readout of the line your cursor is on:

● ● ● *The status bar — a live syllable count*

[Editor] ♪ 8 syl · 12/4

| First frost

♪ 8 syl is the syllable count of the current line, updated as you type; 12/4 is your position — line 2 of a four-line stanza. It answers, without a command, the question a metrical poet asks a hundred times an hour: **is this line at its ten? is this haiku line at its five?** It counts in your project's language — exact for Russian, honestly approximate for English (Chapter 2's limit, carried to the desk).

The next stanza, in one keystroke

When a stanza is done and you want the next, press **Ctrl+B Shift+Y**. Inkhaven creates a sibling verse paragraph of the **same** type, right after this one, and opens it for editing — so a sonnet grows stanza by stanza, or a villanelle tercet by tercet, without your hands leaving the flow. (Structure only, of course — the new stanza is empty; the lines are yours to write.)

Read the poem back

With a form declared, ask the Poet what it sees. Press **F** for the fast track — a deterministic, offline scan of metre and rhyme. Its findings land in the Output pane:

● ● ● *Ctrl+B ♪ → P → F — findings in the Output pane*

Output —————

♪ Praise	Line 1 scans cleanly as iambic pentameter.
♪ Note	Line 2 scans short: 8 of 10 syllables.
♪ Concern	Lines 2↔4 (B–B): "temperate" / "boom" — do not rhyme.

Three severities, and the whole philosophy in them (Chapter 5): **Praise** for a line that keeps its promise, **Note** for a departure worth seeing, **Concern** for a promise plainly broken. The harshest word is Concern, and even that is a flag, never a fix. What you do about it is yours: mend the rhyme, or keep it and mean it.

For a reading a ruler cannot give — where an image strains, whether the sonnet's argument turns at its volta — press **E** to engage the slow track. It sends the stanza to a language model under a strict observer's brief and returns **prose to weigh**, in the poem's own language, headed **♪ Inner Poet**. It never rewrites your poem; there is nothing to accept, only something to consider.

Let it read as you go — ambient

Pressing **F** on every stanza gets old. Press **A** instead and turn on **ambient**: now every verse paragraph with a declared form is scanned the moment you open it, its findings posted without a keystroke. Because the fast track uses no model, ambient is free — there is no cost to it and no cap on it, only a quiet guard so identical lines aren't re-scanned. Move through a manuscript of poems and each one greets you with its own reading.

Silence what you're keeping on purpose

A poet breaks rules deliberately, and a reader that keeps flagging a rule you have **chosen** to break is a reader you stop trusting. So a finding can be **suppressed**: silence its key (its severity, kind, and line), and the fast track and ambient mode stop reporting it. The scan still counts it — nothing is hidden from the totals — but the Output pane goes quiet about it, and the status line notes how many you have silenced (**3 findings (1 suppressed)**). Your decisions persist across sessions, kept in a small **inner_poet.db** beside your project; Inkhaven remembers what you have already answered.

The whole book at a glance

Open the Outline (**Ctrl+2**, or **Ctrl+B Shift+0**) and a book of verse shows you its progress without your opening a single poem:

● ● ● *Ctrl+2 — the Outline, with completion chips*

```
Outline _____
▼ Sonnets
  ▼ 1 – First frost
    J First frost           8/14
  ▼ 2 – The thaw
    J The thaw             14/14 ✓
  ▼ 3 – (untitled)
    J a beginning         3/14
```

Each verse row carries its glyph and a completion chip — 8/14 while a sonnet drafts, 14/14 ✓ when it is whole. Land on one and the detail panel names its form, its **written / expected** count, and any structural issue the form checker found — a villanelle’s missing refrain, a sonnet without its turn. The whole manuscript’s state, read off one screen.

Source beside translation

If the stanza is a translation — a **para:verse-translation** paragraph holding a source and its rendering, split by a **---** line — press **T** for the two-column view:

● ● ● *Ctrl+B J → P → T — the translation view*

```
≠ Translation _____
| source (ru)           | → translation (en)
| Мой дядя самых честных... | → My uncle, of the purest...
| — trilemma (ru → en) —
| Form ██████████ 62% 1/1 lines keep the foot
| Meaning ██████████ the AI axis – engage (E)
| Sound ██████████ 80% alliteration 0.17 → 0.14
```

Source and translation side by side, line-aligned, with the trilemma of Chapter 7 measured beneath: **Form** and **Sound** counted, **Meaning** left blank because it is judgement, not a number — and the pane points you to the Poet’s slow track for

it. The source language is detected for you; the whole thing is read-only, a review, never a rewrite.

The loop, entire

That is the desk. You made a stanza and declared its form; you wrote it with the count in view and grew it a keystroke at a time; you read it back — instantly with **F**, reflectively with **E**, continuously with ambient — and silenced what you meant to keep; you saw the whole book's progress in the Outline and weighed a translation side by side. At no point did Inkhaven write a word of your poem, judge a line good or bad, or cross from measuring into making. The ruler stayed a ruler. The poem stayed yours. That is the only promise this book has, and it holds all the way to the last keystroke.

WHAT YOU LEARNED

- The whole editor loop lives under **Ctrl+B J → P — F** fast-scan, **E** engage, **D** declare a form, **T** translation view, **A** ambient.
- Make a verse paragraph (**i / t** in the Tree → **J**), then **declare its form** with **D** — the one act the desk adds over the CLI, writing a language-localised poem: sidecar so the tools have a target.
- Write with the live status readout (**J N syl · l L/M**); grow the poem a stanza at a time with **Ctrl+B Shift+Y**.
- Read back with **F** (Praise / Note / Concern to Output) or **E** (an AI reading); turn on **ambient** (**A**) to scan as you navigate; **suppress** a finding you are keeping on purpose — the decision persists in `inner_poet.db`.
- See every poem's completion in the Outline (**J 8/14, 14/14 ✓**); review a translation side by side with **T**. Throughout, Inkhaven measures and reports — it never writes the poem.

APPENDIX A



The poetry workflow

Every command, key, and script-word used in this book, in one place. The commands share the shape `inkhaven poetry <action> [options]`; add `--help` to any of them for the full signature. Throughout, `--language` (or `-l`) takes one of `en`, `ru`, `fr`, `de`, `es` and defaults to `en`.

Forms — declaring intent

poetry forms List the eighteen built-in forms with one-line descriptions.

poetry forms --form <name> [--language L] Print a form's poem: block, localised to the language (Russian adds `allow_pyrrhic` / `require_final_stress`, and so on).

poetry forms --new [--name M] Scaffold a form: custom block to edit and paste onto a poem or into `.inkhaven/custom-forms.hjson`.

Measuring — sound, metre, rhyme

poetry syllabify <word> [--language L] Break a word into syllables (cuts from *hypher*, count from vowel nuclei), marking the stressed syllable with `'`.

poetry syllabify --line "..." [--language L] The same, word by word, over a whole line.

poetry metre --line "..." [--language L] Scan a line into `/` (stressed) `×` (unstressed) `·` (flexible) marks, name the detected metre, and report a conformance.

poetry metre --line "..." --form <name> [--language L] Also check the line against the form's **declared** metre — syllable count, fit, and any feminine-ending or catalectic tag.

poetry rhyme <w1> <w2> [--language L] Classify a rhyme — quality (perfect / near / eye / none), type (masculine / feminine / dactylic), and the shared tail; normalised by the language's own rules (German devoicing, French mute-e, Russian akanye).

The Inner Poet — reading a stanza

poetry scan --text "..." --form <name> [--language L] The fast track — findings over every line, each with a severity (Praise / Note / Concern), kind (Metre / Rhyme), and message.

poetry scan ... --json The same findings as a structure, with a top-level `concerns` count.

poetry scan ... --fail-on-concern Exit non-zero if any Concern is present — a CI gate. Opt-in; stops only for Concerns.

poetry status --text "..." --form <name> [--language L] [--json] Completion — the `written` / `expected` line count, the `drafting` /

complete state, and any structural issues (a villanelle's refrains, a sonnet's volta).

Translation

```
poetry trilemma --source "..." --translation "..." [--form <name>]
[--language L] [--to-language L2]
```

Score a verse translation on **form** (metre + rhyme fidelity) and **sound** (alliteration/repetition texture); the **meaning** axis is left blank — engage the Inner Poet's slow track for it.

In the editor

Ctrl+B J Open the Inner-reader overview.

P (in the overview) Open the Inner Poet.

F (Inner Poet) Fast check the poem paragraph under the cursor — findings to the Output pane (offline, instant).

E (Inner Poet) Engage the slow track — a bounded LLM reading, returned as a thought. Advisory only; never edits the poem.

Poems live as paragraphs of the **para:verse-*** family — **verse-line** (||), **verse-stanza** (J), **verse-couplet** (≡), **verse-tercet** (.), **verse-quatrain** (∴), **verse-translation** (⇔) — whose glyphs show a poem's shape in the Tree and Outline panes.

In Bund

All read-only (**store_read**); each takes its arguments off the stack and pushes a value or dictionary. Every **ink.poem.X** also answers to the shorter **poem.X**.

<word> <lang> ink.poem.syllable_count → the syllable count (an integer).

<line> <lang> ink.poem.scan_line
→ { pattern, syllables, metre }.

<w1> <w2> <lang> ink.poem.rhyme → { quality, type, shared }.

```
<text> <form> <lang> ink.poem.status
```

```
→ { complete, lines, expected, issues }.
```

Run a one-off with `inkhaven bund '<code>'`; add `--help` to any command in this book for its full option list.

AFTERWORD

About the author



Vladimir Ulogov.

Vladimir Ulogov has spent decades building infrastructure for distributed systems — the kind of software that watches other software. Early in his career he worked on monitoring and telemetry platforms; later years took him into federated observability, telemetry buses, and the architecture of systems that have to make sense of millions of data points without losing the thread.

Observability, in the end, is a discipline of **coherence** — of never reporting a state the system cannot account for, of insisting that every signal follow from something real. It is not an accident that a tool he built for writers carries the same instinct into the poetry engine this book describes: it measures a line and never invents a verdict

it cannot account for — and it never writes the poem.

What makes him slightly unusual in his corner of the industry is a tendency to write his own tools — not small utilities, but programming languages. The Bund language (its compiler, its VM, its document store, its parser) lives in a long series of Rust crates on crates.io. `rust_dynamic`, `rust_multistackvm`, `bundcore` — each is a building block that exists because the off-the-shelf options didn't fit the shape of the work. Inkhaven's poetry layer grew the same way: the metre engine written to scan invented languages turned out, with no extra machinery, to scan real verse just as well.

A work of love

Inkhaven is open source, under a permissive licence — you can read it, fork it, study it, modify it, and pass it on. Strictly speaking the licence also lets you sell it; the author would, gently but firmly, disagree with your doing that. Inkhaven was not designed as a *for sale* project. It is a work of love made for the people who can least afford to pay for software — the novelist on a battered laptop, the graduate student writing a thesis, the field linguist documenting a language with no budget at all — and turning it into a commercial product would betray the reason it exists. It carries no analytics, no telemetry, no upsell; the binary will never phone home.

A note on cooperation

Vladimir believes firmly in the human capacity for mutual help — that we make better work, and live better lives, when we share what we know and what we build. Open source is one of the most concrete expressions of cooperation our era has produced: code read, improved, and passed forward without payment, without permission, by people who will never meet. If this book helps you hear a poem a little more clearly — one of your own, or one you are reading closely for the first time — that is enough.

Where to find more

GitHub [@vulogov](#) — the source for Inkhaven, Bund, and the dozen-plus Rust crates that carry the infrastructure. Issues and pull requests welcome.

LinkedIn [/in/vladimirulogov](#) — posts on observability, the occasional long-form essay.

YouTube [@vulogov](#) — talks and walkthroughs from the conference trail.

*If the tool ever gets in the way of the poem instead of out of it, open an issue on GitHub.
The author reads them.*